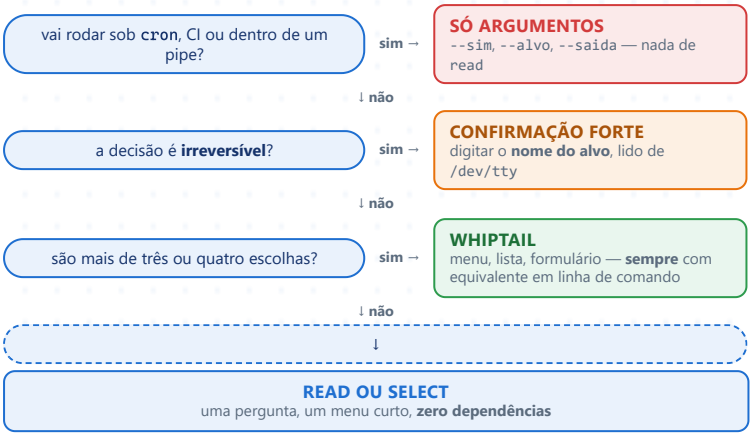
# INTERAÇÃO: MENUS, CORES E CAIXAS DE DIÁLOGO

*O script de limpeza era cuidadoso: mostrava o tamanho e pedia `read -p "Confirma? [s/N] "` antes do `rm -rf`. Funcionou por meses — até alguém escrever `gerar_lista.sh | limpar.sh`. O `read` leu a **primeira linha da lista** em vez do teclado. Era `srv-web-01`, não bateu com `s`, e foi só sorte. **O defeito não é o `read`: é supor que existe alguém do outro lado.** Script bem escrito verifica se há interação e se comporta de forma segura quando não há.*

**076**
GUIAS LINUX

## QUE INTERFACE USAR · E A ANATOMIA DA COR




ANATOMIA DE UM ESCAPE ANSI




`0` normal · `1` negrito · `4` sublinhado
`31` vermelho · `32` verde · `33` amarelo
`34` azul · `36` ciano · `37` branco · `41`–`47` as mesmas no fundo
`\e[0m` **sempre** no fim — sem ele a cor vaza para o resto do terminal

**Prefira `tput` a escrever a sequência à mão.** `tput setaf 1`, `tput bold` e `tput sgr0` consultam o terminfo do terminal em uso e devolvem **string vazia** quando ele não suporta cor — a proteção acontece sozinha. Escrever `\e[1;31m` direto funciona em "quase" todo lugar, e esse "quase" inclui o arquivo de log, o `cron` e o terminal de quem usa leitor de tela.

**A pergunta do topo da árvore é a que mais economiza retrabalho:** se o script pode acabar num pipe ou num agendador — e quase todo script acaba —, a interatividade tem de ser **opcional**. Escreva primeiro a versão com argumentos e só depois acrescente o modo interativo que preenche os mesmos valores; o contrário, começar pelo menu e tentar automatizar depois, custa uma reescrita. **As três guardas antes de emitir cor** são o destino ser um terminal (`[[ -t 1 ]]`), a variável `NO_COLOR` não estar definida e o `TERM` não ser `dumb`. Se qualquer uma falhar, as variáveis de cor viram string vazia e **o resto do script não muda em nada** — a cor é opcional em **tempo de execução**, não em tempo de escrita. O mesmo `[[ -t 1 ]]` guarda a barra de progresso: sem ele, um log de `cron` acumula milhares de linhas de `\r`. Para progresso de um **fluxo de dados**, o `pv` já faz tudo isso sem uma linha de código.

## READ, SELECT E WHIPTAIL NA PRÁTICA

kali@kali: ~/lab-interacao

```
# leia do TERMINAL, não do stdin — e falhe alto se não houver
$ read -rp 'Confirma? [s/N] ' r < /dev/tty
$ interativo() { [[ -t 0 && -t 1 ]]; }
# sem terminal: RECUSE, não assuma. --sim libera o modo automático
$ printf 's\n' | ./seguro.sh; echo "status: $?"
erro: exige confirmacao; use --sim para modo nao interativo
status: 1
# o irreversível não aceita "s": exige o nome por extenso
$ read -rp "Digite o nome do alvo para confirmar: " c < /dev/tty
$ [[ "$c" == "$ALVO" ]] || { echo 'cancelado.' >&2; exit 1; }
# cor sob guarda tripla: desligada, tudo vira string vazia
$ if [[ -t 1 && -z "${NO_COLOR:-}" && "${TERM:-dumb}" != dumb ]]; then
    C_RESET=$(tput sgr0); C_VERM=$(tput setaf 1)
  else C_RESET='' C_VERM=''; fi
$ erro() { printf '%s[ERRO ]%s %s\n' "$C_VERM" "$C_RESET" "$*" >&2; }
# menu sem dependência nenhuma; case sobre $REPLY, não sobre o texto
$ PS3='Escolha uma opção: '
$ select o in "Listar" "Reiniciar" "Sair"; do case "$REPLY" in …
# whiptail desenha no stdout e devolve pelo stderr: troque os três
$ escolha=$(whiptail --menu "Tarefa:" 15 60 4 \
      "1" "Atualizar" "2" "Limpar" 3>&1 1>&2 2>&3); echo $?
0   # 0 confirmou · 1 cancelou · 255 pressionou Esc
```

READ: AS OPÇÕES QUE IMPORTAM

| | |
|---|---|
| `-r` | não interpreta a barra invertida — **sempre** |
| `-p 'txt'` | prompt antes de ler; sai no `stderr`, para não poluir o `stdout` que outro programa pode consumir |
| `-s` | não ecoa o digitado (senha) |
| `-n N` | retorna após N caracteres, sem esperar Enter |
| `-t N` | desiste após N segundos, status > 128 — **obrigatório** em prompt que possa travar script automatizado |
| `-e -i val` | edição de linha com valor inicial editável |
| `-a arr · -u N` | lê para um array · lê do descritor N |

WHIPTAIL: AS CAIXAS

| | |
|---|---|
| `--yesno` | pergunta; status **0** sim, **1** não, **255** Esc — tratar os três é o que evita que um Esc vire confirmação |
| `--inputbox` | uma linha de texto · `--passwordbox` sem eco |
| `--checklist` | múltipla escolha com `ON`/`OFF` |
| `--gauge` | barra alimentada por percentuais na entrada |
| `--stdout` | dispensa a ginástica `3>&1 1>&2 2>&3` nas versões recentes |

O CHECKLIST DA INTERAÇÃO

- ✓ prompt e diagnóstico em `stderr`, **resultado em `stdout`** — é o que faz `r=$(script.sh)` funcionar
- ✓ confirmação lida de `/dev/tty`, nunca do `stdin`
- ✓ **Enter vazio nunca confirma** operação destrutiva
- ✓ modo não interativo **explícito** (`--sim`, `--batch`) e recusa clara sem terminal
- ✓ toda pergunta tem equivalente em **opção de linha de comando**

**A "confirmação fantasma" é o modo mais silencioso de disparar um comando destrutivo.** Num pipe, num `cron` ou num contêiner sem TTY, `read -p "Confirma? [s/N]"` não espera ninguém: consome a próxima linha do `stdin`. Se ela for `s`, o `rm -rf`, o `mkfs` ou o `cryptsetup luksFormat` rodam **sem que ninguém tenha digitado nada**. As defesas: ler de `< /dev/tty` (que **falha** quando não há terminal, em vez de aceitar dados como resposta); `[[ -t 0 && -t 1 ]]` antes de perguntar, recusando com mensagem em vez de assumir; padrão sempre "não"; e, para o irreversível, exigir **o nome do alvo por extenso** — com tamanho e modelo mostrados na pergunta, para dar para conferir. **E nunca receba senha por argumento:** `argv` aparece no `ps` para todos os usuários da máquina e no histórico do shell. Use `read -rs`, arquivo com permissão `0600` ou variável de ambiente, nessa ordem.

## RESUMO RÁPIDO

- ✓ **`read` sem terminal lê do pipe, não do teclado.** Confirmação sempre com `< /dev/tty`, que falha alto quando não há terminal em vez de aceitar uma linha de dados como resposta.
- ✓ **`[[ -t 0 ]]` e `[[ -t 1 ]]` detectam interatividade.** Sem ela, recuse e exija uma opção explícita (`--sim`, `ASSUMIR_SIM=1`).
- ✓ **O padrão nunca é o destrutivo.** Enter vazio cancela; para o irreversível, exija o nome do alvo por extenso.
- ✓ **Prompt e diagnóstico em `stderr`, resultado em `stdout`** — é o que permite capturar a saída do script com `$( )`.
- ✓ **`select` resolve o menu simples sem dependência**, com `PS3` como prompt e `case` sobre `$REPLY`, não sobre o texto.
- ✓ **Cor é opcional em tempo de execução:** `tput` em vez de escape à mão, sob a guarda de `[[ -t 1 ]]`, `NO_COLOR` e `TERM` ≠ `dumb`. Desligada, as variáveis viram vazias e nada mais muda.
- ✓ **`whiptail` e `dialog` escrevem a interface no `stdout` e o resultado no `stderr`.** A troca `3>&1 1>&2 2>&3` (ou `--stdout`) captura a escolha; os status 0, 1 e 255 distinguem confirmar, cancelar e `Esc`.
- ✓ **Toda pergunta precisa de equivalente em opção de linha de comando.** Script que só funciona com menu não pode ser agendado, testado nem reproduzido.

## NO KALI

**Confirmação forte em tudo que é irreversível.** `cryptsetup luksFormat`, `mkfs`, `dd if=… of=/dev/sdX` e a limpeza de resultados de teste apagam o que não volta: exigir que a pessoa **digite o nome do dispositivo**, e não `s`, é a diferença entre um erro de digitação e um disco perdido — e o alvo tem de aparecer **por extenso na pergunta**, com tamanho e modelo, para que dê para conferir. **O `whiptail` é o que o `kali-tweaks` usa** — e é o mesmo motor do `dpkg-reconfigure`, então já está instalado em qualquer Debian ou Kali sem acrescentar dependência; se você escreve utilitário para a equipe, seguir a convenção deixa a experiência coerente com o resto do sistema. **Nunca peça senha com eco ligado nem a receba por argumento:** `read -rs` para digitar, arquivo com permissão `0600`, ou variável de ambiente, nessa ordem de preferência — senha em `argv` aparece no `ps` para todos os usuários e no histórico do shell, e num teste autorizado isso é **vazamento de credencial do cliente**. **Cor no relatório é armadilha:** saída de ferramenta capturada com `$( )` traz as sequências de escape junto e a comparação de string falha por um motivo invisível — desligue a cor na origem (`--no-color`, `NO_COLOR=1`) ou limpe com `sed -r 's/\x1b\[[0-9;]*m//g'` antes de processar. **E script de ataque tem de ser automatizável:** um menu bonito que exige teclado impede que a ação seja repetida de forma idêntica, e a **reprodutibilidade** é o que sustenta o relatório de um teste autorizado — se não dá para rodar o mesmo comando outra vez e obter o mesmo resultado, não dá para provar o que foi feito nem para o cliente verificar a correção depois. **Família RHEL:** o `whiptail` (pacote `newt`) costuma existir nas duas famílias, mas o utilitário de referência ali é o `dialog`, e `--gauge` e `--checklist` têm diferenças de sintaxe — detecte com `command -v whiptail || command -v dialog`. O `tput` funciona igual, com a ressalva de que o console do RHEL em modo texto pode oferecer menos cores: consultar `tput colors` antes de assumir 256 é a checagem correta.